

[Contact \(https://login2explore.com/#contact\)](https://login2explore.com/#contact)

JPDB API Command Reference

Language Syntax

All JPDB syntax follows JSON Syntax Rules.

- <...> - User specified value.
- <<...>> - Optional parameter, may take default value if not used.
- | - OR, One of the specified values must be chosen.
- ... - Ellipsis, indicating more values can be inserted if required.

Setup Instructions

Connection Token - Connecting to JPDB API.

- Connection-token is used to interact with JsonPowerDB, it must be included in every request of jsonPowerDB for security purpose.

How to get the 'connection-token' from JsonPowerDB :

- Login to JPDB API Dashboard (<http://api.login2explore.com:5577/user/index.html>).
- Click on Tools (http://api.login2explore.com:5577/user/user_tools.html) option on left navigation .
- Select **Tokens** panel and then select 'connection-token' option from dropdown ,
- Copy the 'connection-token' from the Token table.
- And if there is no 'connection-token' in the token table or if you want to generate a new 'connection-token', click on the **Generate Connection_Token** button as It will generate a new-connection token.
- Copy the newly generated 'connection-token' from token table.

Response Headers in JPDB :

One or more of the following information may be returned in the response, all of which give time in milliseconds unit. All information will be returned with IML, IRL, IDL, ISL APIs.

- serverTime** - Time between receiving request and returning response by JsonPowerDB Server.
- reqResTime** - Time taken to convert request to reponse by the API (not in the KVP API). It's the combination of parseTime and execTime.
- parseTime** -Time taken to parse and validate the request.
- execTime** - Actual time of executing the validated request.

Commands

IML (JPDB Index Manipulation Language) - To insert, update and delete Json data.

Http method : POST

Base url : <http://api.login2explore.com:5577>

End-point url : /api/iml (mentioned in command when different)

PUT : Insert single record in the database

SET : Insert a new record or update an existing record based on primary key

[Contact \(https://login2explore.com/#contact\)](https://login2explore.com/#contact)

SET_ALL : Insert and update records based on primary key

REMOVE : Remove records from the database

Pluggable APIs:-

If the any of the following pluggable commands are not working in your instance, please email with details at support@login2explore.com

COPY COLUMN : It will copy columns

RENAME COLUMN : It will rename columns name

REMOVE COLUMN : It will remove columns

CHANGE COLUMN TYPE : It will change columns data type

MERGE COLUMN : It will merge two columns.

IRL (JPDB Index Retrieval Language) - To retrieve Json data.

Http method : POST

Base url : <http://api.login2explore.com:5577>

End-point url : /api/irl

GET (Deprecated - use GET_BY_KEY) : Retrieve single record by json data

GET_BY_KEY : Retrieve single record by json data

GET_ALL : Retrieve multiple records from relation

FIND_RECORD / FIND_ALL_RECORDS : Retrieve records by json data

GET_RECORD (Deprecated - use GET_BY_RECORD): Retrieve single record by record number

GET_BY_RECORD : Retrieve single record by record number

FIRST_RECORD : Retrieve first record from the JPDB database

LAST_RECORD : Retrieve last record from the JPDB database

NEXT_RECORD : Retrieve next record by record number

PREV_RECORD : Retrieve previous record by record number

GET_RELATION_STATS / GET_RELATION_SIZE : Get total number of records and size of relation

GET_ALL_DB : Retrieve all database names from JPDB database

GET_ALL_RELATION (Deprecated - use GET_ALL_RELATIONS): Retrieve all relation details from JPDB database

Contact (https://login2explore.com/#contact)

GET_DB_STATS / GET_ALL_RELATIONS : Retrieve all relation details from JPDB database

GET_USER_STATS : Retrieve user details.

GET_INDEXED_COL : Retrieve all indexed column names from JPDB database

GET_NON_INDEXED_COL : Retrieve all non indexed column names from JPDB database

GET_ALL_COL : Retrieve all column names from JPDB database

IS_COLUMN_INDEXED : Check whether a column is indexed or not

IS_COLUMN_EXIST : Check whether a column exist or not in the JPDB database

IS_RELATION_EXISTS : Check whether a Relation exist or not in the JPDB database

IS_DATABASE_EXISTS : Check whether a Database exist or not in the JPDB database

SYNC_DB : Returns delta rows from multiple relations of a database from a given timestamp

IDL (JPDB Index Definition Language) - To add index column and remove column / relation / database.

Http method : POST

Base url : http://api.login2explore.com:5577

End-point url : /api/idl

ADD_INDEX (Deprecated - use INDEX_COLUMN): Convert a non index column to index column

INDEX_COLUMN : Convert one or more non-indexed columns to indexed columns

REMOVE_INDEX (Deprecated - use UNINDEX_COLUMN): Convert a index column to non index column

UNINDEX_COLUMN : Convert one or more indexed columns to non-indexed columns

REMOVE_RELATION : Remove a relation from the database

REMOVE_DB : Remove a database from the JPDB database

Http method : POST

Base url : http://api.login2explore.com:5577

Contact (https://login2explore.com/#contact)

For reference : Watch the tutorial (https://www.youtube.com/watch?v=eMUq5wuPtow)

Ping API for checking whether JsonPowerDB is online or not

ping

pluggable ping

Send email API / Get unique ID API

send_email : Send emails from JsonPowerDB

get_unique_id : It will generate a unique Id

Counter API

For reference : Watch the tutorial (https://www.youtube.com/watch?v=Qr6tabt34hM)

INIT : Initializes a counter

GET : Retreives the value of counter

INC : Increments the value of counter

Session API

get_new_session : Generate a new session token

validate_session : Validate a session token

remove_session : Remove a session token

Statistical Analysis API - To calculate the aggregate commands

Http method : POST

Base url : http://api.login2explore.com:5577

End-point url : /jpdb/serverless/stats/aggregate

aggregate

Syntax

Req

Resp

Contact (https://login2explore.com/#contact)

```
.....<<"reqId":"req-id",>> LF
.....<<"key":"key-value",>> LF
.....<<"forceAggr":"boolean-value",>> LF
.....<<"dbName":"database-name",>> LF
.....<<"rel":"relation-name",>> LF
.....<<"keys":["key-1",<"key-2">,<"key-N">],>>|<"key-col" LF
.....<<"data":["value-1",<"value-2">,<"value-N">],|<"data-col" LF
.....<<"aggr":[optional-list-of-aggr-commands] LF
....} LF
}
```

There are two variation of this aggregate function:-

- Firstly, we can directly give the data array and key array in "data" and "key" attribute.
- Secondly, we can fetch the data from database.
In this case we have to give DB name("dbName") and Relation name("rel") of the database and in "data" attribute, give the column name that contains data and in the "key" attribute give the column name that contains the corresponding key.

reqId : Optional, it is used for multiple request of same data (Streaming data/Batch processing). If there are multiple requests from the same 'reqId' then it will calculate the values according to the aggregate command of the first request.

removeReqId : Optional, It can be true or false (by default removeReqId is false).

- If removeReqId is false then data/calculation remains there in memory and we can continue the batch processing .
- If removeReqId is true then it remove the particular batch req id data/calculation after processing the current aggregation

forceAggr : Optional, It can be true or false (by default forceAggr is false).

- If forceAggr is false then only Long and Double data types are allowed.
- If forceAggr is true then it auto convert other data types into Double.

dbName : It contains database name (should be given when we want the data from database).

rel : It contains relation name (should be given when we want the data from database).

keys : Optional, if keys are present then it will show the keys correspond with their data values in the response. Keys can be given in string format if we want the keys from database or in array format with following type in it

- Long
- Double
- String
- Boolean
- null

data : It is the values upon which the aggregate will be performed. We have to give column name if we want the data from database otherwise directly values in an array. The values can be

- Long
- Double
- String (if forceAggr is true then only acceptable otherwise consider as error.)
- Boolean (if forceAggr is true then only acceptable, true as 1 and false as 0 otherwise consider as error.)
- null (if forceAggr is true then only acceptable otherwise consider as error.)

aggr : It is the commands to be calculated. Aggregate commands are :

- SUM** : It will give addition of all the elements present in the data.
- COUNT** : It will give the number of element present in the data.
- AVG** : It will give the average of all the elements present in the data.
- MIN** : It will give minimum element present in the data (if the minimum element is not unique in the data then it will give all keys correspond with that element).
- MAX** : It will give maximum element present in the data (if the maximum element is not unique in the data then it will give all keys correspond with that element).
- MEAN** : It will give mean of all the element present in the data.
- MEDIAN** : It will give median of all the element present in the data.
- MODE** : It will give mean of all the element present in the data.
- TOP:N** : It will give the top 'N' elements present in the data with their keys, grouping all keys with the same value (where N can be any long value).
- BOTTOM:M** : It will give the bottom 'M' elements present in the data with their keys, grouping all keys with the same value (where M can be any long value).
- RANK:P** : It will rank the elements present in the data with their keys up to 'P', with tied values sharing the same rank and the next rank being skipped (where P can be any long value).
- RANGE** : It will give the range between maximum element and minimum element present in the data.

Contact (https://login2explore.com/#contact)

JsonPowerSQL

JsonPower Query - To retrieve data from database.

Http method : POST

Base url : http://api.login2explore.com:5577

End-point url : /api/irl

select

Syntax

Req

Resp

```
{
  ...."token":.<"connection-token">,
  ...."select":.{
    .....<<"rows":.<"ALL">,>>
    .....<"cols":.<."*">|.<[
    .....{
    .....<"alias1">.<"col1">|.<[.relate-expr1.>
    .....}
    .....<<,{
    .....<"alias2">.<"col2">|.<[.relate-expr2.>
    .....}>>...
    .....]>
    .....},
    .....<<"createTime":.<boolean-value>,>>
    .....<<"updateTime":.<boolean-value>,>>
    .....<"from":.<"main-relation">,
    .....<<"where":.{
    .....[.<[condition1]>.<<,[condition2],...[conditionN]>>.>
    .....<<,.<[condition1]>.<<,[condition2],...[conditionM]>>.>>...
    .....]>|.<<[condition]>>>,>
    .....<<"limit":.<[row-count,.offset]>>>
  }
}
```

from : Relation name from which records will be retrieved. Relation name is in the format : db_name | rel_name.

cols : Columns which you wants to retrieve.

- If column is belongs to the same relation which is given in the **from** then, Columns name is in the format : { "alias_name" : "column-name" }.
- Or You can provide column name like this : { "alias_name" : "db_name | main-rel_name . col_name" }.

relate-expr : Relate Expression Will retrieve column from another relation. Relate expression is an array which is in the format:

- { "alias_name" : ["db_name | rel_name . col_name2Retrieve" ,
"db_name | main-relation . relate-coln" ,
"db_name | rel_name . col_name2Relate"]
}

- At index 0, Column you want to retrieve from different relation, must be in the format : db_name | rel_name . col_name .
- At index 1, Index Column must be from same relation as given in **from** for comparing its value with another column value, It is in the format : "db_name | main-relation . relate-coln" , or you can only provide : column-name but It is only applicable at index 1.

- [Syntax](#)

Contact (https://login2explore.com/#contact)

`[[ [condition1], [condition2], ...[conditionM] ] ] ... ]`

The above syntax will work in the form : `[ [ [condition1] AND [condition2] AND ...[conditionN] ] OR [ [condition1] AND [condition2] AND ...[conditionM] ] OR ... ]`

For this type of condition you can write condition in this way :

EX. : (a OR b) AND (c OR d) you can write this condition as : a AND c OR a AND d OR b AND c OR b AND d

- **Simple type-** `[ condition ]`

- **Condition**

- `[ "col-name", "<" | "<=" | "=" | ">=" | ">" | "!=" , { "value" : value1 } ]`
- `[ "col-name", "in" | "!in", [value1, value2, value3 ...] ]`
- `[ "col-name", "between", [value1, value2] ]`

- At index 0, column-name must be from same relation as given in **from** whose value is to be compared with **value** / column. It is in the format : `db_name|main-rel_name.col_name`
- At index 1, It must be a operator to perform comparison.
- At index 2, value you want to compare with column present at index 0 and **value** must be given in the JSONObject.
- At index 2, for "in" and "!in" operator jsonArray is provided having one or more values to specify multiple values in a WHERE clause.
- At index 2, for "between" operator only two values are provided in jsonArray which are used as begin and end values.

limit : Limit will allows you to retrieve only certain number of records.

- **row-count** : row-count is the number of records you want to retrieve from relation.
- **offset** : offset allows you to skip a specified number of rows from the beginning of the result set. If offset is 6, row-count is 5 and total 20 rows in result-set, it will return 5 rows starting from 7th row till 11th row.

NoSQL Reference

NoSQL is a collection of java methods. Basic knowledge of java language is required to write NoSQL scripts.

'this' - Object of class JsonDBExecuteQuery, used to return records to client. Must be passed in methods as 'this' if specified.

finalQueryResult - Object of class QueryResultSet, used to display the resultant json records to the client.

`QueryService.getRecordsByDynamicJoin()` : Returns a record of main primary relation inside all its linked relation data.

`QueryService.getRecordsByDynamicJoin( )` : Returns a record of main primary relation inside all its linked relation data.

`QueryService.get( )` : Return all the record-numbers matching to a given key

`QueryService.get( )` : Returns one row

`QueryService.getAll( )` : Returns all the record number matching to given key in provided range

QueryService.getRecordSize() : Returns number of rows in the given relation

Contact (https://login2explore.com/#contact)

QueryService.getProject() : Return QueryResultSet containing all the json

QueryService.selectAll() : Return numbers of records within a given range from a relation

QueryService.compare() : compare the objects

QueryService.compareIgnoreType () : Compare two objects of different types

QueryService.equals() : Compare for the equality of two objects

QueryService.equalsIgnoreType() : Compare for the equality of two objects of different types

QueryService.exists() : Check whether one object is exists in another object or not

How to add a record in finalQueryResult Set

How to get json object from a record

How to get response message of a query

How to get status of a query

How to get query execution time of a query

How to set response message of a query

How to set status of a query

How to convert given datatype to a Record

ISL (JPDB Index Scripting Language) - To execute NoSQL script.

Http method : POST

Base url : http://api.login2explore.com:5577

End-point url : /api/isl

EXECUTE_SCRIPT : Execute NoSQL query code

REMOVE_SCRIPT : Remove NoSQL query script

GET_SCRIPT_NAMES : Retrieve all NoSQL script names

JPDB kvp Api Command Reference

Http method : POST

Base url : <http://api.login2explore.com:5577>

Contact : <https://login2explore.com/#contact>

End-point url : /kvp/api/hm

For reference : Watch the tutorial (https://www.youtube.com/watch?v=I_uciHERh_w)

hmPut : Insert Hash Map data in the default or given workspace.

hmGet : Get the HashMap data from default or given workspace

hmExists : Check whether the given HashMap key is exists in the default or given workspace

hmDel : Remove single key of hashmap from default or given workspace

hmClear : Clears given HashMap from default or given workspace

removeHm : Removes given HashMap from default or given workspace

hmPutX : Add the expiry time to the hashmap key, after the expiry time, key will not be accessed

hmGetX : Add the expiry time to the HashMap key, after the expiry time, key will not be accessed

hmExistsX : Add the expiry time to the HashMap key, after the expiry time, key will not be accessed

hmX : Add the expiry time to the HashMap key, after the expiry time, key will not be accessed

hmNx : Removes the expiry time from the given HashMap key, after the expiry time, key will be accessed

String – To insert, get and delete String data.

Http method : POST

Base url : <http://api.login2explore.com:5577>

End-point url : /kvp/api/str

dfPut : Insert single key-value data in the default or given workspace

dfGet : Get the given key value from default or given workspace.

dfExists : Check whether a the given key exists in the default or given workspace.

dfDel : Remove single key from the default or given worksapce

dfClear : Clear the data from the default or given workspace

Bulk – To execute bulk request.

Http method : POST

Base url : <http://api.login2explore.com:5577>

End-point url : /kvp/api/bulk

Bulk Request

Http method : GET

Base url : <http://api.login2explore.com:5577>

Contact (https://login2explore.com/#contact)

End-point url : /kvp/api/any

getAll : Returns all the data of a selected data structure of a selected workspace.

selectNextWs : It returns a name of newly created workspace and set it to the current workspace.

selectWs : Set the given workspace to the current workspace.

Meta - To get the meta data of the KVP-DB.

Http method : POST

Base url : <http://api.login2explore.com:5577>

End-point url : /kvp/api/meta

getAllWs : Returns all the workspace name.

getAllDataStructures : Returns all the data structure names and their respective data type of the current workspace or the given workspace.

getWsName : Returns the workspace name of given workspace ID.

getWsId : Returns the workspace ID of the given workspace name.

JPDB GeoSpatial API

JPDB GeoSpatial API

Http method : POST

Base url : <http://api.login2explore.com:5577>

End-point url : /api/irtl/geo

For reference : Watch the tutorial (<https://www.youtube.com/watch?v=8ipBwzy8DmU>)

GEO_DISTANCE : From a given list of keys based on latitude and longitude columns returns distance(s), record number(s), angle(s), coordinate(s) and other required column(s)

GEO_DISTANCE : For a given time period of a given value of a column optionally returns distance(s), record number(s), angle(s), coordinate(s) and other required column(s)

GEO_DISTANCE(Serverless) : For a given list of coordinates optionally returns distance(s), record number(s), angle(s), coordinate(s) and other required column(s)

AREA_CIRCLE(https://login2explore.com/#contact) : For a given centre coordinate and a list of coordinates returns optionally distance(s), record number(s), angle(s), coordinate(s) and other required column(s) from within the circular (circle / inner and outer square) area of a given radius

AREA_CIRCLE(Serverless) : For a given centre coordinate and a list of coordinates returns optionally distance(s), record number(s), angle(s), coordinate(s) and other required column(s) from within the circular (circle / inner and outer square) area of a given radius

Drive API

Drive API – To upload, download files in drive.

Http method : POST

Base url : http://api.login2explore.com:5577

Pluggable APIs:-

If the following commands are not working in your instance, it means this pluggable API is not available in this instance of JsonPowerDB. Please contact support@login2explore.com

uploadtext : Upload text files into the drive

uploadbinary : Upload image files as Base-64 data into the drive

downloadtext : Download text files from the drive

downloadbinary : Download image files as Base-64 data from the drive

rename : Rename files and folders in drive

copy : Copy files from one folder to another in drive

move : Move files from one folder to another in drive

remove : Remove files from the drive

fileinfo : Get detailed information regarding file / folder

listfiles : List files present in a folder from the drive

listfolders : List folders present in a folder from the drive

foldersize (Deprecated – use FILE_INFO): Display the size of folder in the drive

Import and Export API

Import API – To upload csv files as relation.

Http method : POST

Base url : http://api.login2explore.com:5577

Pluggable APIs:-

If the following commands are not working in your instance, please contact Login2explore.com

import (csv file) : Import csv file in database relation

import (json file) : Import json file in database relation

Export API – To download file from database.

Http method : POST

Base url : http://api.login2explore.com:5577

Pluggable APIs:-

If the following commands are not working in your instance, please contact Login2explore.com

export (json file) : Export database relation in json file.

jpdb-commons.js

Javascript Library

Collection of JavaScript functions to speedup the development.

Click to Explore

Contact (https://login2explore.com/#contact)

URL::"http://login2explore.com/jpdb/resources/js/0.0.3/jpdb-commons.js (http://login2explore.com/jpdb/resources/js/0.0.3/jpdb-commons.js)

Click to Explore

JPDBObject.js

Synchronized JPDBObject

- A javascript tool to synchronize the JsonPowerDB database for client, salient features are:
- # **Synchronized** -keep the data at client localStorage in sync with the JPDB database.
 - # **AI** - intelligent enough to sync data as per individual client requirements.
 - # Allows to keep multiple instance of JPDBObject, each on different JsonPowerDB database.
 - # Easy and fast for application development.
 - # Seamless experience for developers to code for JsonPowerDB databases.

URL::"http://login2explore.com/jpdb/resources/js/0.0.3/JPDBObject.js (http://login2explore.com/jpdb/resources/js/0.0.3/JPDBObject.js)

Click to Explore

Contact (https://login2explore.com/#contact)

```
// JPDB Object will sync relations from database LF
//excludeArr - Store all relation name that was given by user in relation APP-SYNC-EXCLUDE-RELATION from JPDB datab
function·JPDBObject(pUniqueDbId,·pDbName,·pRefreshTime,·pRelIncludeArr,·pConnToken,·pDbBaseUrl){·/* ... */·} LF
LF
// It will assign value of parameter to this and create object of all relation in pRelIncludeArr LF
function·JPDBObject(pUniqueDbId,·pDbName,·pRefreshTime,·pRelIncludeArr,·pConnToken,·pDbBaseUrl){·/* ... */·} LF
LF
// After every refresh time JPDBObject fetch records from database LF
// DEFAULT VALUE = 2000 (2 sec) MIN VALUE = 500 (0.5 sec) LF
// If prefreshTime < 500 it automatically sets Refresh Time to 500 LF
function·setRefreshTime(prefreshTime){·/* ... */·} LF
LF
// After every Avail Refresh Time usage count of availArr gets updated LF
// DEFAULT VALUE = 60000 * 60 * 6 (6 hrs) LF
function·setAvailRefreshTime(pAvailArrRefreshTime){·/* ... */·} LF
LF
// Using setModeAuto() JPDBObject will decide which relations to sync automatically LF
function·setModeAuto(){·/* ... */·} LF
LF
// Using setModeManual JPDBObject will sync relations provided by user LF
function·setModeManual(){·/* ... */·} LF
LF
// Set optimumSize and maximumSize of localStorage capacity LF
// DEFAULT optimumSize = 1000 * 1000 * 2 (2 MB) LF
// DEFAULT maximumSize = 1000 * 1000 * 3 (3 MB) LF
function·setCapacityLS(pOptimumSize,·pMaximumSize){·/* ... */·} LF
LF
// Set Optimization technique to sync relation LF
// Currently two optimization technique are available SIZE and USAGE LF
// DEFAULT optimizationTechnique = SIZE LF
function·setOptimizationTechnique(pOptimizationTechnique){·/* ... */·} LF
LF
// Start execution of synching data in localStorage. LF
function·syncDB(){·/* ... */·} LF
LF
// To get name of database LF
function·getDbName(){·/* ... */·} LF
LF
// Add relation in includeArr and relObjArr - used to sync specified relations in JPDB Object LF
function·addRelToSync(pRelInclude){·/* ... */·} LF
LF
// It will remove relation from includeArr (relObjArr) - used to sync specified LF
// relations in JPDBObject LF
function·removeRelFromSync(pRelRemove){·/* ... */·} LF
LF
// Count total number of relations in sync or to be sync LF
function·getRelCount(pRelObjArr){·/* ... */·} LF
LF
// Get all relation name present in relObjArr(sync) LF
function·getAllRelName(){·/* ... */·} LF
LF
// It will give all exclude relations name from server. LF
function·getAllExcludeRelNames(){·/* ... */·} LF
LF
// GetRecordByRecNo - It will retrieve single record by record number from localStorage LF
function·getRecordByRecNo(pRelName,·pRecNo){·/* ... */·} LF
LF
// GetNextRecord - It will retrieve next record by record number from localStorage LF
function·getNextRecord(pRelName,·pRecNo){·/* ... */·} LF
LF
```

Contact (https://login2explore.com/#contact)

```
// It will retrieve first record from relation (LocalStorage) LF
function·getFirstRecord·(pRelName)·{·/* ... */·} LF
LF
// It will retrieve last record from relation (LocalStorage) LF
function·getLastRecord(pRelName)·{·/* ... */·} LF
LF
// Retrieve single record by json data from LocalStorage LF
function·getRecord(pRelName,·pColName,·pvalue)·{·/* ... */·} LF
LF
// GetGetAllRecord - It will retrive multiple record from relation(LocalStorage) LF
function·getGetAllRecord·(pRelName,·pColNameArr,·timestamp,·pPageSize,·pPageNumber)·{·/* ... */·} LF
LF
// Retrieve all records by json data from LocalStorage LF
function·findRecord·(pRelName,·pColName,·pvalue)·{·/* ... */·} LF
LF
// It will give delete_count and record_count of relation LF
function·getRelSize·(pRelName)·{·/* ... */·} LF
LF
// It will return true or false for col Exist or not LF
function·isColExist(pRelName,·pColName)·{·/* ... */·}
```

Using JPDBObject is very simple. The following example is self explanatory.

Syntax

Req

```
// The following must be the first line to create the JPDBObject instance. LF
var·jpd01·=·new·JPDBObject(pUniqueDbId,·pDbName,·pRelIncludeArr,·pConnToken,·pDbBaseUrl); LF
LF
// The following lines are optional. If not provided JPDBObject will take default values. LF
jpd01.setRefreshTime(prefreshTime); LF
jpd01.setAvailRefreshTime(pAvailArrRefreshTime); LF
jpd01.setOptimizationTechnique(pOptimizationTechnique); LF
jpd01.setModeAuto()/jpd01.setModeManual(); LF
jpd01.setCapacityLS(pOptimumSize,·pMaximumSize); LF
LF
// The following must be the last line to initiate the sync of data with JsonPowerDB LF
jpd01.syncDB();
```